Why Messaging

stop making every service wait for every other

Watch the 4-min video

In one line: a message broker lets a service publish work or events and move on, while consumers process them independently; you gain decoupling, buffering for spikes, resilience to slow or failed consumers and easy fan-out, and you pay with eventual consistency, duplicates and harder debugging.

1 Checkout is only as fast as the slowest call

When an order is placed, the order service calls the email service, the analytics service and the inventory service, one after another. If email is slow, checkout is slow. If analytics is down, checkout fails, even though the customer doesn't care about analytics. Every new feature adds another call to the chain.

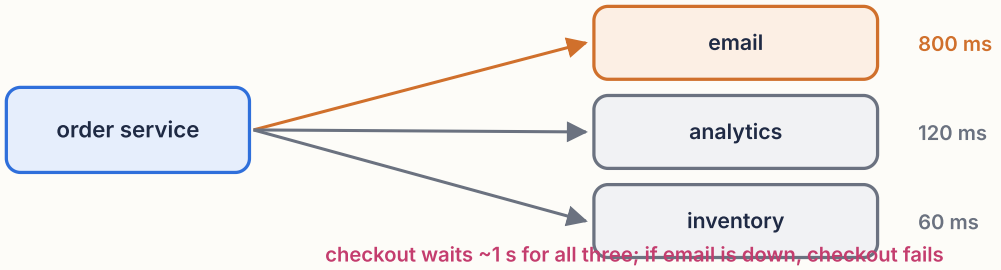


Figure 1. Synchronous calls: latencies add up, and any failure fails the whole request.

2 Put a broker in between

With **messaging**, the order service publishes one message, `OrderPlaced`, to a **broker** (Kafka, RabbitMQ, SQS, Pub/Sub...) and returns to the customer. The broker stores the message durably, and each interested service **consumes** it when it's ready. The producer doesn't know or care who the consumers are.

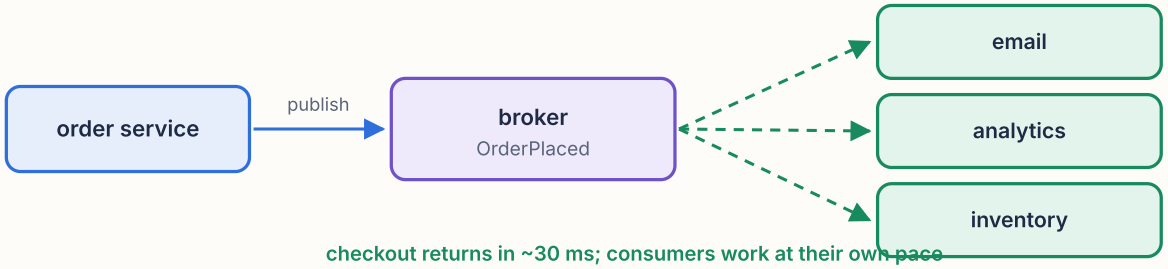


Figure 2. Asynchronous messaging: the producer publishes once; consumers process independently.

WORKED EXAMPLE: ORDERPLACED

The order service saves the order and publishes one event, `OrderPlaced`. The email service consumes it and sends a confirmation. Analytics records it. Inventory reserves the stock. On Black Friday, orders arrive ten times faster than email can send; the messages queue up and email catches up over the next twenty minutes, while checkout stays fast. When the email service is deployed or crashes, nothing upstream notices. Next month a fraud service subscribes to the same event, with no change to the order service.

3 What you gain



Figure 3. The four big benefits of messaging.

4 What it costs

- **Eventual consistency:** the email arrives a bit later; the stock is reserved after the response. Design the UX for it.
- **Duplicates and ordering:** most brokers deliver at least once; consumers must be idempotent (lesson 20) and handle order carefully (lesson 29).
- **Operations:** a broker to run, monitor lag, dead-letter queues (lesson 31).
- **Debugging:** a flow spread across services needs correlation IDs and tracing (lesson 40).

Keep synchronous calls when the caller needs the answer now (checking a price, authorising a payment) and use messages for work that can happen after the response.

COMMON MISTAKES

- Saving to the database and then publishing, so a crash in between loses the event (use an outbox, lesson 37).
- Using messages for request/response where the user waits anyway.
- No monitoring of consumer lag.

5 Key terms

Message broker: stores and delivers messages between services

Producer / publisher: sends messages

Consumer / subscriber: receives and processes messages

Asynchronous: the sender doesn't wait for processing

Consumer lag: how far behind a consumer is

Fan-out: one message to many consumers

6 Interview questions

> **Q1. Why introduce a message broker between services?** BASIC
> To decouple them: the producer publishes and returns, consumers process independently, spikes are buffered, and a slow or failed consumer doesn't break the producer.

Q2. What are the downsides of asynchronous messaging? BASIC

Eventual consistency, duplicate and out-of-order messages, more infrastructure to run, and harder end-to-end debugging.

Q3. Checkout calls five services and is slow. What do you move to messaging? SCENARIO

Everything the customer doesn't need before the response: emails, analytics, loyalty points, search indexing. Keep payment authorisation and stock checks synchronous if the response depends on them.

Q4. An email provider outage takes checkout down. Fix it. SCENARIO

Publish OrderPlaced and let the email service consume it; checkout succeeds and emails go out when the provider recovers.

Q5. When is synchronous better than messaging? TRICKY

When the caller needs the result to continue, like validating a coupon or authorising a card, or when the extra latency and complexity aren't justified.

Q6. What happens if the producer crashes right after saving the order but before publishing? TRICKY

The event is lost and consumers never hear about the order. Use the transactional outbox so saving and publishing are tied together.

Next: SD-I-002 · Message Queues vs Event Streams backendroadmap.com